

# Guia de Apresentação - Form Builder

Roteiro prático para demonstrar o desafio Full Stack Developer 2026, explicar arquitetura, decisões técnicas, QA e próximos passos.

**Produção:** <https://criador-de-form.fly.dev>

**Repositório:** <https://github.com/TauanGramacho/criador-de-form>

**Data da revisão:** 24/08/2026

## Frase central para abrir a apresentação

Este projeto é um MVP full stack de criação, publicação e acompanhamento de formulários. O usuário autenticado administra apenas os próprios formulários; o formulário publicado fica disponível por URL pública para qualquer pessoa responder; as respostas são salvas no banco e analisadas na área administrativa.

MVP aqui não significa protótipo incompleto. Significa produto mínimo viável com o fluxo essencial fechado: autenticação, criação, publicação, resposta pública, persistência, indicadores e deploy.

| Fluxo de demo          | O que mostrar                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------|
| 1. Login               | Entrar com e-mail/senha ou Google OAuth e mostrar que a área administrativa exige sessão.                |
| 2. Criar formulário    | Criar título, descrição e campos como Nome e E-mail.                                                     |
| 3. Configurar campos   | Explicar rótulo, tipo e obrigatório. Campos avançados foram removidos para manter a experiência simples. |
| 4. Publicar            | Publicar o formulário e copiar a URL pública gerada pela aplicação.                                      |
| 5. Responder sem login | Abrir a URL pública em outra aba e enviar uma resposta como visitante.                                   |
| 6. Ver indicadores     | Voltar ao painel do dono e mostrar respostas, dashboard e respostas individuais.                         |

# Arquitetura

A aplicação segue uma arquitetura simples e defensável: React no cliente, Go como backend único e banco relacional para persistência. O frontend não conversa diretamente com o banco.

| Camada          | Responsabilidade                                                                                                            | Tecnologia                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Frontend        | Interface administrativa, formulário público, dashboard e consumo da API tipada.                                            | React, TypeScript, Vite, MUI, React Router, TanStack Query |
| Contrato de API | OpenAPI gerado pelo backend e usado para gerar o client TypeScript.                                                         | Huma + @hey-api/openapi-ts                                 |
| Backend         | Autenticação, sessão, regras de dono, CRUD de formulários, publicação, validação de respostas e entrega do SPA em produção. | Go, Huma, chi, cookies HttpOnly                            |
| Persistência    | Usuários, sessões, formulários, campos e respostas.                                                                         | SQLite local;<br>Postgres/Supabase em produção             |
| Deploy          | Build multi-stage e execução do backend servindo API e frontend.                                                            | Docker, Fly.io, Supabase Postgres                          |

## Ponto importante para banca

Todas as chamadas de dados passam pelo backend em Go. O React usa a API HTTP; o banco fica protegido atrás do servidor. As rotas administrativas usam o usuário da sessão e filtram por owner\_id.

```
server run --config=./config.yaml
server run --address=localhost:8080
ADDRESS=localhost:8080 server run
```

Isso atende ao requisito de configuração por arquivo, argumentos e variáveis de ambiente. Não precisa ser apenas local: o mesmo desenho funciona em produção usando variáveis e secrets.

## Organização das pastas

| Pasta/arquivo                      | Como explicar                                                                         |
|------------------------------------|---------------------------------------------------------------------------------------|
| api/                               | Contrato OpenAPI versionado. Serve de ponte entre backend e client TypeScript gerado. |
| backend/cmd/server/                | Entrada da CLI do servidor. Centraliza comandos como run e migrate.                   |
| backend/internal/app/api.go        | Registro das rotas HTTP: auth, formulários, publicação, respostas e API pública.      |
| backend/internal/app/auth.go       | Sessões, cookies, hash de senha e Google OAuth.                                       |
| backend/internal/app/store.go      | Camada de persistência. Aplica filtros por dono e conversa com SQLite/Postgres.       |
| backend/internal/app/validation.go | Validação dos campos do formulário e das respostas públicas.                          |
| backend/migrations/                | Criação das tabelas e índices do banco.                                               |
| frontend/src/App.tsx               | Rotas e telas principais da aplicação React.                                          |
| frontend/src/api/generated/        | Client TypeScript gerado automaticamente a partir do OpenAPI.                         |
| scripts/                           | Scripts auxiliares de geração, deploy e configuração de secrets.                      |
| supabase/migrations/               | Migração equivalente para o banco Postgres hospedado no Supabase.                     |
| Dockerfile e fly.toml              | Empacotamento e configuração do deploy em produção.                                   |

## Tecnologias e justificativa

| Escolha            | Por que faz sentido no desafio                                                                                           |
|--------------------|--------------------------------------------------------------------------------------------------------------------------|
| Go no backend      | Atende ao requisito obrigatório de backend Go e entrega uma API standalone, performática e simples de empacotar.         |
| Huma               | Gera OpenAPI automaticamente a partir das rotas e tipos Go, reduzindo inconsistência entre documentação e implementação. |
| React + TypeScript | Permite uma UI administrativa produtiva, com tipagem e organização clara para o client gerado.                           |
| MUI                | Entrega componentes consistentes e rápidos para um MVP com aparência profissional.                                       |
| TanStack Query     | Cuida de cache, carregamento, atualização e invalidação das chamadas da API.                                             |
| SQLite local       | Facilita rodar em qualquer máquina sem instalar servidor de banco.                                                       |
| Supabase Postgres  | Banco relacional em produção, fácil de inspecionar e compatível com deploy online.                                       |
| Fly.io             | Hospeda o backend Go e o frontend estático juntos, mantendo um único domínio para API e aplicação.                       |

### Como explicar a decisão SQLite + Supabase

SQLite foi escolhido para facilitar execução local e cumprir a ideia de aplicação multiplataforma. Em produção, a mesma camada Store usa Postgres/Supabase por DATABASE\_URL. Assim, o projeto roda simples localmente e também tem banco real online.

## Requisitos do desafio atendidos

| Requisito                                              | Status na aplicação                                                                                   |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Usuário acessa a aplicação e realiza login             | Atendido com e-mail/senha e Google OAuth.                                                             |
| Área administrativa autenticada                        | Atendido. Rotas privadas exigem cookie de sessão.                                                     |
| Criar novo formulário                                  | Atendido. CRUD de formulário pelo painel.                                                             |
| Adicionar e configurar campos                          | Atendido com rótulo, tipo, obrigatório, opções para select e validação.                               |
| Publicar formulário                                    | Atendido. Publicação muda status para published e gera slug/URL pública.                              |
| URL pública respondível por qualquer pessoa            | Atendido. Rotas públicas não exigem login.                                                            |
| Respostas armazenadas e consultadas pelo administrador | Atendido. Respostas ficam ligadas ao formulário e são exibidas ao dono.                               |
| Validação no backend                                   | Atendido em validation.go: campos obrigatórios, tipos, e-mail, número, select e chaves desconhecidas. |
| OpenAPI e client gerado                                | Atendido. Backend gera OpenAPI e frontend usa client TypeScript gerado.                               |
| Configuração flexível                                  | Atendido por arquivo, flags e variáveis de ambiente.                                                  |

## Segurança, isolamento e QA

Cada usuário autenticado administra apenas os próprios formulários. Aqui, administrador significa o dono autenticado dos formulários que criou, não um superadmin global.

| Controle     | Como foi tratado                                                                                                          |
|--------------|---------------------------------------------------------------------------------------------------------------------------|
| Sessão       | Cookie HttpOnly, SameSite=Lax e Secure em produção.                                                                       |
| Senha        | Hash com bcrypt. Cadastro exige confirmação de senha e não loga automaticamente após criar conta.                         |
| Google OAuth | Client ID/secret ficam no backend; redirect configurado para produção.                                                    |
| IDOR/BOLA    | Rotas privadas chamam requireUser e consultam por owner_id. Outro usuário recebe 404 ao tentar acessar formulário alheio. |
| Form público | Somente published aparece por slug público. O público pode responder, mas não vê área administrativa.                     |
| Validação    | Backend rejeita respostas com campo desconhecido, tipo errado ou campo obrigatório vazio.                                 |
| Banco        | Frontend não acessa Supabase diretamente. A DATABASE_URL fica como secret do Fly.io.                                      |

### Ação antes da entrega final

A senha do Supabase apareceu no chat. Por segurança, rotacione a senha no painel do Supabase e atualize o secret DATABASE\_URL no Fly.io usando scripts/set-fly-database-secret.ps1. Depois faça deploy ou reinicie a aplicação para o secret entrar em uso.

### QA final em produção

| Teste                                   | Resultado                                                  |
|-----------------------------------------|------------------------------------------------------------|
| Usuário A cria formulário               | Criado formulário temporário frm_ym9-gvDyM5VBysNUsMyxK03z. |
| Usuário B lista formulários             | Não viu o formulário do usuário A.                         |
| Usuário B acessa direto /api/forms/{id} | Recebeu HTTP 404.                                          |
| Link público                            | API pública abriu o formulário publicado.                  |
| Envio público de resposta               | Resposta criada: rsp_xr9ouCc_ub8Vgu9V6Eaf6KYN.             |
| Dono consulta respostas                 | Usuário A visualizou 1 resposta.                           |
| Limpeza                                 | Formulário de QA foi apagado após o teste.                 |

## Como responder perguntas prováveis

| Pergunta                                              | Resposta curta                                                                                                                        |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Isso é um MVP?                                        | Sim. É um MVP funcional com o fluxo principal completo, deploy e banco real em produção.                                              |
| Por que qualquer usuário logado parece administrador? | Porque o desafio fala em área administrativa. Aqui cada conta é administradora dos próprios formulários, com isolamento por owner_id. |
| Por que o formulário público não pede login?          | Porque o requisito diz que qualquer pessoa com a URL pública pode preencher.                                                          |
| O frontend acessa o banco?                            | Não. O React chama a API; a API Go valida, autentica e persiste.                                                                      |
| Onde entra OpenAPI?                                   | O backend gera o contrato; o frontend gera o client TypeScript a partir dele.                                                         |
| Como roda localmente?                                 | Backend com Go e SQLite; frontend com Vite. SQLite não exige servidor.                                                                |
| Como roda online?                                     | Docker no Fly.io, DATABASE_URL como secret e Supabase Postgres como banco.                                                            |
| Quais seriam próximos passos?                         | Exportar CSV, permissões por equipe, templates, rate limit persistente, logs/auditoria e testes e2e com Playwright.                   |

## Roteiro verbal de 2 minutos

1. 'Eu implementei um criador de formulários full stack em Go e React, com autenticação, painel administrativo, publicação pública e dashboard de respostas.'
2. 'O backend é a fonte de verdade: valida campos, controla sessões, filtra tudo por owner\_id e gera o OpenAPI usado pelo frontend.'
3. 'Localmente o projeto usa SQLite para facilitar execução. Em produção usa Supabase Postgres, sem expor o banco no frontend.'
4. 'Vou demonstrar o fluxo principal: login, criação do formulário, publicação, resposta pública sem login e análise das respostas pelo dono.'
5. 'Também validei o isolamento: um segundo usuário não enxerga nem acessa o formulário do primeiro.'

### Fechamento sugerido

'Eu priorizei entregar o fluxo completo pedido no desafio, com contrato de API gerado, client tipado, deploy real e isolamento entre usuários. Como próximos incrementos, eu adicionaria exportação de dados, colaboração entre usuários e testes e2e automatizados.'